Advanced Statistical Methods II

Computing the LS Estimator in Non-linear Regression

1 The computational problem

We observe $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$ and fit the model $y_i = f(\mathbf{x}_i; \boldsymbol{\theta}) + \varepsilon_i$, $\boldsymbol{\theta} \in \mathbb{R}^p$. The non-linear least-squares estimator (NLSE) is

$$\widehat{\boldsymbol{\theta}} \in \arg\min_{\boldsymbol{\theta}} S(\boldsymbol{\theta}), \qquad S(\boldsymbol{\theta}) = \sum_{i=1}^{n} \{y_i - f(\mathbf{x}_i; \boldsymbol{\theta})\}^2 = \|\mathbf{y} - \mathbf{f}(\boldsymbol{\theta})\|^2,$$

and an interior minimizer satisfies the normal equations $\mathbf{J}(\widehat{\boldsymbol{\theta}})^\top \{\mathbf{y} - \mathbf{f}(\widehat{\boldsymbol{\theta}})\} = \mathbf{0}$. In most non-linear models these equations have no closed-form solution, so $\widehat{\boldsymbol{\theta}}$ has to be computed by an *iterative* algorithm. Every method in this handout has the same template:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + t_k \, \mathbf{d}^{(k)}, \qquad k = 0, 1, 2, \ldots, \tag{1}$$

where $\boldsymbol{\theta}^{(0)}$ is a starting value, $\mathbf{d}^{(k)}$ is a *direction* and $t_k > 0$ is a *step length*. Three questions have to be answered: which direction, how far, and when to stop. The methods differ in how much they use about S: its slope only (gradient descent, Section 3), its slope and curvature (Newton's method, Section 4), or the special least-squares structure of S (the Gauss–Newton method, Section 5). Every method also needs a starting value, which is where *self-starting models* come in (Section 6).

1.1 Ingredients: the gradient and the Hessian of S

Write $f_i(\boldsymbol{\theta}) = f(\mathbf{x}_i; \boldsymbol{\theta})$ and $r_i(\boldsymbol{\theta}) = y_i - f_i(\boldsymbol{\theta})$ for the ith residual, so that $S(\boldsymbol{\theta}) = \sum_i r_i(\boldsymbol{\theta})^2$. Recall the $n \times p$ Jacobian

$$\mathbf{J}(\boldsymbol{\theta}) = \left(\frac{\partial f_i(\boldsymbol{\theta})}{\partial \theta_j} \right)_{i,j}.$$

The gradient. For $j = 1, \ldots, p$, by the chain rule,

$$\frac{\partial S}{\partial \theta_j} = \sum_{i=1}^{n} 2\, r_i \, \frac{\partial r_i}{\partial \theta_j} = -2 \sum_{i=1}^{n} r_i \, \frac{\partial f_i}{\partial \theta_j}.$$

The jth entry of $\mathbf{J}^\top \mathbf{r}$ is $\sum_i (\partial f_i / \partial \theta_j)\, r_i$, so

$$\nabla S(\boldsymbol{\theta}) = -2\, \mathbf{J}(\boldsymbol{\theta})^\top \mathbf{r}(\boldsymbol{\theta}). \tag{2}$$

The Hessian. Differentiate $\partial S/\partial\theta_j = -2\sum_i r_i\,\partial f_i/\partial\theta_j$ once more with respect to θ_l, using the product rule and $\partial r_i/\partial\theta_l = -\partial f_i/\partial\theta_l$:

$$\frac{\partial^2 S}{\partial\theta_j\partial\theta_l} = -2\sum_{i=1}^{n}\left\{\frac{\partial r_i}{\partial\theta_l}\frac{\partial f_i}{\partial\theta_j} + r_i\frac{\partial^2 f_i}{\partial\theta_j\partial\theta_l}\right\} = 2\sum_{i=1}^{n}\left\{\frac{\partial f_i}{\partial\theta_j}\frac{\partial f_i}{\partial\theta_l} - r_i\frac{\partial^2 f_i}{\partial\theta_j\partial\theta_l}\right\}.$$

The first sum is the (j,l) entry of $\mathbf{J}^{\mathsf{T}}\mathbf{J}$. Writing $\nabla^2 f_i$ for the $p\times p$ matrix of second derivatives of f_i,

$$\nabla^2 S(\boldsymbol{\theta}) = 2\Bigg(\underbrace{\mathbf{J}^{\mathsf{T}}\mathbf{J}}_{\text{first derivatives only}} - \underbrace{\sum_{i=1}^{n} r_i\,\nabla^2 f_i}_{\text{needs second derivatives}}\Bigg). \tag{3}$$

In a linear model $\mathbf{f} = \mathbf{X}\boldsymbol{\beta}$, so $\mathbf{J} = \mathbf{X}$ and $\nabla^2 f_i = \mathbf{0}$, and (3) reduces to $2\mathbf{X}^{\mathsf{T}}\mathbf{X}$. The second term in (3) is small when the residuals are small *or* when the model is nearly linear. Remember this: it is the key to the Gauss–Newton method.

With one parameter ($p = 1$) write $J_i = \partial f_i/\partial\theta$ and $f_i'' = \partial^2 f_i/\partial\theta^2$. Then (2) and (3) read

$$S'(\theta) = -2\sum_{i=1}^{n} J_i\,r_i, \qquad S''(\theta) = 2\sum_{i=1}^{n}\left(J_i^2 - r_i\,f_i''\right). \tag{4}$$

1.2 Descent directions

Definition 1. A vector $\mathbf{d}$ is a *descent direction* for S at $\boldsymbol{\theta}$ if $\nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{d} < 0$.

Lemma 1. *If $\mathbf{d}$ is a descent direction at $\boldsymbol{\theta}$, then $S(\boldsymbol{\theta} + t\mathbf{d}) < S(\boldsymbol{\theta})$ for all sufficiently small $t > 0$.*

Proof. By a first-order Taylor expansion, $S(\boldsymbol{\theta} + t\mathbf{d}) = S(\boldsymbol{\theta}) + t\,\nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{d} + o(t)$ as $t \downarrow 0$. Dividing by t,

$$\frac{S(\boldsymbol{\theta} + t\mathbf{d}) - S(\boldsymbol{\theta})}{t} = \nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{d} + \frac{o(t)}{t} \longrightarrow \nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{d} < 0,$$

so the left-hand side is negative for all small enough $t > 0$. □

A descent direction guarantees progress only for a *small enough* step. How small is the business of the step length t_k.

2 Two running examples

Each method is tried on the same examples, so that the methods can be compared. All numbers below were produced by the R code printed with them; the code of each example runs on its own, in the order shown.

2.1 A one-parameter example

A substance decays, and the fraction remaining is measured after $x = 1, 2, 3$ hours: $y = (0.70,\ 0.30,\ 0.20)$. We fit the first-order decay model

$$f(x;\theta) = e^{-\theta x}, \qquad \theta > 0.$$

With one parameter everything can be computed by hand:

$$J_i = \frac{\partial f_i}{\partial \theta} = -x_i\, e^{-\theta x_i}, \qquad f_i'' = \frac{\partial^2 f_i}{\partial \theta^2} = x_i^2\, e^{-\theta x_i}.$$

Every J_i is negative: increasing the decay rate lowers every fitted value, and lowers the later ones relatively more.

```
## fraction of a substance remaining after 1, 2 and 3 hours
x <- c(1, 2, 3); y <- c(0.70, 0.30, 0.20)
f   <- function(th) exp(-th * x)              # mean vector f(theta)
J   <- function(th) -x * exp(-th * x)         # Jacobian: df_i/dtheta
f2  <- function(th) x^2 * exp(-th * x)        # second derivatives d2f_i/dtheta2
S   <- function(th) sum((y - f(th))^2)                          # sum of squares
dS  <- function(th) -2 * sum(J(th) * (y - f(th)))               # S'(theta)
d2S <- function(th) 2 * sum(J(th)^2 - (y - f(th)) * f2(th))     # S''(theta)
th_hat <- optimize(S, c(0, 3), tol = 1e-12)$minimum
c(theta.hat = th_hat, S.min = S(th_hat))
## everything at theta = 1, observation by observation
th <- 1
round(data.frame(x, y, f = f(th), r = y - f(th), J = J(th), f2 = f2(th)), 4)
```

```
 theta.hat      S.min
0.50614811 0.01382577
  x   y      f      r       J     f2
1 1 0.7 0.3679 0.3321 -0.3679 0.3679
2 2 0.3 0.1353 0.1647 -0.2707 0.5413
3 3 0.2 0.0498 0.1502 -0.1494 0.4481
```

The minimum is at $\widehat{\theta} = 0.5061$. At $\theta = 1$ all three residuals are positive (the fitted values are too low, because the decay is too fast) and all J_i are negative, so by (4) $S'(1) = -2\sum_i J_i r_i > 0$: the sum of squares increases with θ at $\theta = 1$, and we must move to the left. This is the kind of reasoning that each algorithm automates.

```
par(mfrow = c(1, 2))
tg <- seq(0, 3, length.out = 400)
plot(tg, sapply(tg, S), type = "l", lwd = 2, col = "steelblue",
     xlab = expression(theta), ylab = expression(S(theta)),
     main = "Sum of squares")
abline(v = th_hat, lty = 3)
neg <- tg[sapply(tg, d2S) < 0]
rect(min(neg), -1, 4, 2, col = adjustcolor("firebrick", 0.12), border = NA)
text(2.2, 0.15, "S'' < 0 here", col = "firebrick")
plot(tg, sapply(tg, dS), type = "l", lwd = 2, col = "steelblue",
     xlab = expression(theta), ylab = expression(S * "'" * (theta)),
     main = "Its derivative")
abline(h = 0, v = th_hat, lty = 3)
```

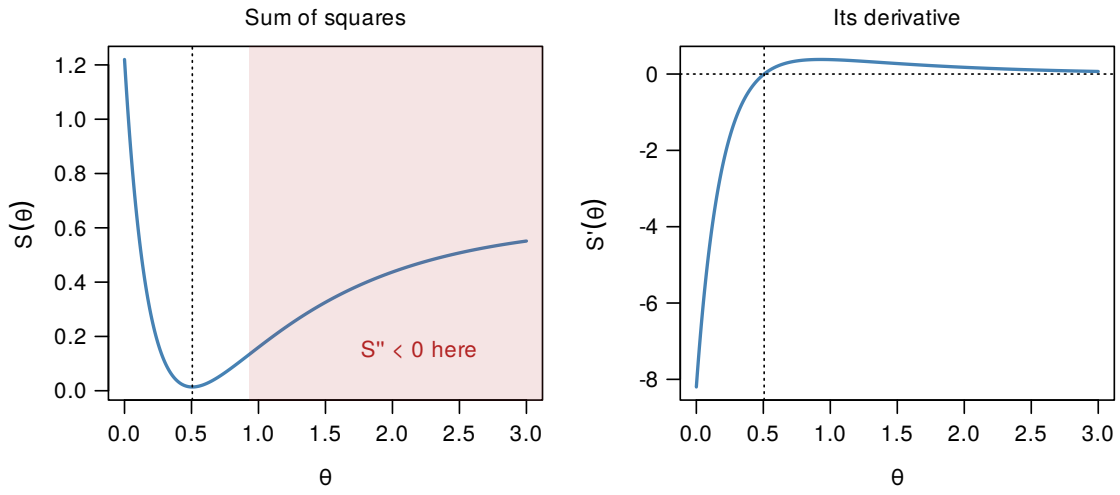


Figure 1: The one-parameter example. Left: $S(\theta)$, with the minimum at $\widehat{\theta} = 0.506$; to the right of $\theta \approx 0.93$ (shaded) $S'' < 0$, so S is not convex. Right: $S'(\theta)$, which tends to zero again as $\theta \to \infty$.

Two features of Figure 1 matter later. First, S is *not convex*: for θ beyond about 0.93 it bends downwards. In (4) the term $-r_i f_i''$ is negative when the residuals are positive and large, and it then outweighs J_i^2. Second, as $\theta \to \infty$ every fitted value tends to 0, S levels off at $\sum_i y_i^2 = 0.62$, and the slope S' tends to zero: far to the right the surface is a *plateau*.

2.2 The Puromycin data

The data. The built-in R data set `Puromycin` comes from an experiment in enzyme kinetics (Treloar, 1974), analysed at length by Bates and Watts (1988). The initial *velocity* (rate, the response) of an enzymatic reaction was measured, in counts of radioactive product per minute per minute, at several concentrations (predictor) of the *substrate*, in parts per million (ppm). The experiment was run on cells treated with the antibiotic puromycin (12 runs) and on untreated cells (11 runs). We use the treated cells: six substrate concentrations, each run twice.

```
pur <- subset(Puromycin, state == "treated")
pur[, c("conc", "rate")]                    # 12 runs: 6 concentrations, 2 each
```

```
   conc rate
1  0.02   76
2  0.02   47
3  0.06   97
4  0.06  107
5  0.11  123
6  0.11  139
7  0.22  159
8  0.22  152
9  0.56  191
10 0.56  201
11 1.10  207
12 1.10  200
```

The model. The rate of an enzymatic reaction rises with the substrate concentration but levels off once the enzyme is saturated. The Michaelis–Menten model describes exactly this:

$$f(x;\boldsymbol{\theta}) = \frac{V_m\, x}{K + x}, \qquad \boldsymbol{\theta} = (V_m, K)^{\mathsf{T}},$$

where x is the substrate concentration (the predictor).

- V_m is the maximum rate: $f \to V_m$ as $x \to \infty$.
- K is the concentration at which the rate is half its maximum: $f(K;\boldsymbol{\theta}) = V_m/2$.
- For $x \ll K$, $f \approx (V_m/K)\, x$, a straight line through the origin; for $x \gg K$, $f \approx V_m$, a constant.

The mean vector is $\mathbf{f}(\boldsymbol{\theta}) = \big(V_m x_i/(K+x_i)\big)_{i=1,\ldots,12}$ and the sum of squares is $S(V_m, K) = \sum_{i=1}^{12}\{y_i - V_m x_i/(K+x_i)\}^2$.

The Jacobian. Row i of $\mathbf{J}(\boldsymbol{\theta})$ holds the two partial derivatives

$$\frac{\partial f_i}{\partial V_m} = \frac{x_i}{K + x_i}, \qquad \frac{\partial f_i}{\partial K} = -\frac{V_m\, x_i}{(K + x_i)^2},$$

and the second derivatives needed by (3) are

$$\frac{\partial^2 f_i}{\partial V_m^2} = 0, \qquad \frac{\partial^2 f_i}{\partial V_m \partial K} = -\frac{x_i}{(K + x_i)^2}, \qquad \frac{\partial^2 f_i}{\partial K^2} = \frac{2V_m\, x_i}{(K + x_i)^3}.$$

Each column of $\mathbf{J}$ is a *sensitivity*: it says how much each fitted value moves when one parameter moves.

- $\partial f/\partial V_m = x/(K+x)$ lies between 0 and 1 and increases with x: changing V_m moves the fitted values most at high concentrations.
- $\partial f/\partial K$ is negative (a larger K means slower saturation, so lower rates) and its size is largest at $x = K$: changing K moves the curve most around the half-saturation point.

The two columns are therefore informative at different concentrations (right panel of Figure 2). The data must cover both regimes for both parameters to be well determined.

The following functions are used by every algorithm below. We compare two starting points that both look reasonable, $A = (250,\ 0.03)$ and $B = (150,\ 0.2)$, written as (V_m, K).

```
x <- pur$conc; y <- pur$rate
f   <- function(th) th[1] * x / (th[2] + x)          # mean vector f(theta)
Jac <- function(th) cbind(Vm = x / (th[2] + x),       # df/dVm
                          K  = -th[1] * x / (th[2] + x)^2)  # df/dK
S     <- function(th) sum((y - f(th))^2)
gradS <- function(th) -2 * drop(crossprod(Jac(th), y - f(th)))
hessS <- function(th) {
  r   <- y - f(th); J <- Jac(th)
  d12 <- -x / (th[2] + x)^2                # d2f / dVm dK   (d2f/dVm2 = 0)
```

```
  d22 <- 2 * th[1] * x / (th[2] + x)^3      # d2f / dK2
  E   <- matrix(c(0, sum(r * d12), sum(r * d12), sum(r * d22)), 2, 2)
  2 * (crossprod(J) - E)
}
A <- c(250, 0.03); B <- c(150, 0.2)         # two starting points (Vm, K)
## reference answer: nls, polished by a few Newton steps
th_hat <- unname(coef(nls(rate ~ Vm * conc / (K + conc), data = pur,
                          start = list(Vm = 200, K = 0.1))))
for (i in 1:5) th_hat <- unname(th_hat - solve(hessS(th_hat), gradS(th_hat)))
round(c(Vm = th_hat[1], K = th_hat[2], S = S(th_hat)), 5)
## the Jacobian at the start A, one row per distinct concentration
round(cbind(conc = x, Jac(A))[c(1, 3, 5, 7, 9, 11), ], 3)
```

```
        Vm          K          S
 212.68374    0.06412 1195.44881
     conc    Vm         K
[1,] 0.02 0.400 -2000.000
[2,] 0.06 0.667 -1851.852
[3,] 0.11 0.786 -1403.061
[4,] 0.22 0.880  -880.000
[5,] 0.56 0.949  -402.183
[6,] 1.10 0.973  -215.365
```

Look at the scales in the last table. The V_m column of $\mathbf{J}$ contains numbers below 1; the K column contains numbers in the hundreds and thousands. This is simply because $V_m \approx 200$ and $K \approx 0.06$ are measured on very different scales, and it will matter a great deal for gradient descent.

```
par(mfrow = c(1, 2))
xg <- seq(0, 1.15, length.out = 300)
mm <- function(x, th) th[1] * x / (th[2] + x)
plot(rate ~ conc, data = pur, pch = 19, ylim = c(0, 260),
     xlab = "substrate concentration (ppm)", ylab = "rate (counts/min/min)",
     main = "Data and two starting curves")
lines(xg, mm(xg, A), lwd = 2, lty = 2, col = "darkorange")
lines(xg, mm(xg, B), lwd = 2, lty = 4, col = "purple")
lines(xg, mm(xg, th_hat), lwd = 2, col = "firebrick")
legend("bottomright", bty = "n", lwd = 2, lty = c(2, 4, 1), cex = 0.85,
       col = c("darkorange", "purple", "firebrick"),
       c("start A = (250, 0.03)", "start B = (150, 0.2)", "least-squares fit"))
## the two columns of the Jacobian, as functions of concentration
Jg <- cbind(xg / (th_hat[2] + xg), -th_hat[1] * xg / (th_hat[2] + xg)^2)
plot(xg, Jg[, 1], type = "l", lwd = 2, col = "steelblue", ylim = c(-1, 1.05),
     xlab = "substrate concentration (ppm)", ylab = "sensitivity",
     main = "Columns of the Jacobian at the fit")
lines(xg, Jg[, 2] / max(abs(Jg[, 2])), lwd = 2, lty = 2, col = "seagreen")
abline(h = 0, lty = 3); abline(v = th_hat[2], lty = 3)
legend(0.3, 0.6, bty = "n", lwd = 2, lty = 1:2, cex = 0.85,
       col = c("steelblue", "seagreen"),
       c("df/dVm", "df/dK (rescaled to [-1, 0])"))
```

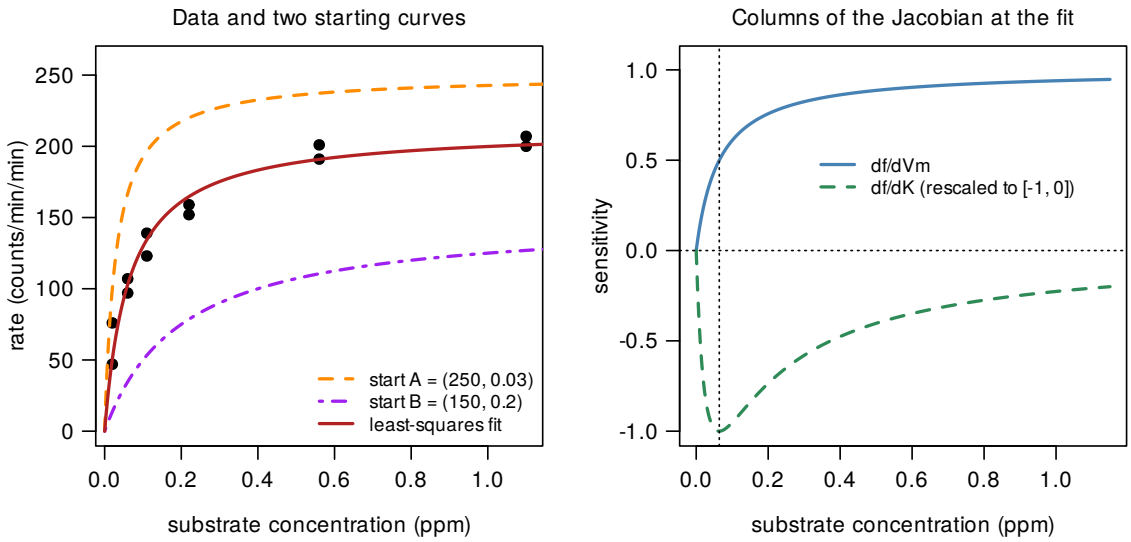


Figure 2: Left: the treated Puromycin runs with the Michaelis–Menten curves at the two starting points and at the least-squares fit. Right: the two columns of the Jacobian at the fit, as functions of concentration; the dotted vertical line is $x = \widehat{K}$, where $|\partial f/\partial K|$ is largest.

Start A has too large a maximum and saturates too early; start B is too low everywhere. Both are the kind of guess one might make from a glance at the data.

3 Gradient descent

3.1 The direction of steepest descent

Which direction decreases S fastest? We answer this in six small steps.

Step 1: compare directions of the same length. A direction $\mathbf{d}$ and the direction $10\mathbf{d}$ point the same way, so only the *direction* matters, not the length. We therefore compare unit vectors $\mathbf{u}$ with $\|\mathbf{u}\| = 1$.

Step 2: the rate of change along u. Fix $\boldsymbol{\theta}$ and a unit vector $\mathbf{u}$, and move along the line $t \mapsto \boldsymbol{\theta} + t\mathbf{u}$. Put $\varphi(t) = S(\boldsymbol{\theta} + t\mathbf{u})$. By the chain rule,

$$\varphi'(0) = \sum_{j=1}^{p} \frac{\partial S(\boldsymbol{\theta})}{\partial \theta_j}\, u_j = \nabla S(\boldsymbol{\theta})^{\mathsf{T}} \mathbf{u}.$$

This number is the *directional derivative* of S at $\boldsymbol{\theta}$ in the direction $\mathbf{u}$: the rate at which S changes as we start to move along $\mathbf{u}$.

Step 3: what we want. By a first-order Taylor expansion,

$$S(\boldsymbol{\theta} + t\mathbf{u}) = S(\boldsymbol{\theta}) + t\,\nabla S(\boldsymbol{\theta})^{\mathsf{T}} \mathbf{u} + o(t).$$

For a small step t the change in S is essentially $t\,\nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{u}$. To decrease S as fast as possible we want $\nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{u}$ to be as *negative* as possible. So the question is: how small can $\nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{u}$ be, over all unit vectors $\mathbf{u}$?

Step 4: the Cauchy–Schwarz inequality. For any two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^p$,

$$|\mathbf{a}^\mathsf{T}\mathbf{b}| \le \|\mathbf{a}\|\,\|\mathbf{b}\|,$$

with equality if and only if one of the vectors is a scalar multiple of the other. *Proof.* If $\mathbf{b} = \mathbf{0}$ both sides are zero. Otherwise, for every real number λ,

$$0 \le \|\mathbf{a} - \lambda\mathbf{b}\|^2 = \|\mathbf{a}\|^2 - 2\lambda\,\mathbf{a}^\mathsf{T}\mathbf{b} + \lambda^2\|\mathbf{b}\|^2.$$

Choose $\lambda = \mathbf{a}^\mathsf{T}\mathbf{b}/\|\mathbf{b}\|^2$, the value that minimizes the right-hand side. Then

$$0 \le \|\mathbf{a}\|^2 - 2\frac{(\mathbf{a}^\mathsf{T}\mathbf{b})^2}{\|\mathbf{b}\|^2} + \frac{(\mathbf{a}^\mathsf{T}\mathbf{b})^2}{\|\mathbf{b}\|^2} = \|\mathbf{a}\|^2 - \frac{(\mathbf{a}^\mathsf{T}\mathbf{b})^2}{\|\mathbf{b}\|^2},$$

which rearranges to $(\mathbf{a}^\mathsf{T}\mathbf{b})^2 \le \|\mathbf{a}\|^2\|\mathbf{b}\|^2$. Equality holds exactly when $\|\mathbf{a} - \lambda\mathbf{b}\| = 0$, that is, when $\mathbf{a} = \lambda\mathbf{b}$.

Step 5: a lower bound for the rate. Apply Cauchy–Schwarz with $\mathbf{a} = \nabla S(\boldsymbol{\theta})$ and $\mathbf{b} = \mathbf{u}$, and use $\|\mathbf{u}\| = 1$:

$$|\nabla S(\boldsymbol{\theta})^\mathsf{T}\mathbf{u}| \le \|\nabla S(\boldsymbol{\theta})\|\,\|\mathbf{u}\| = \|\nabla S(\boldsymbol{\theta})\|, \qquad \text{hence} \qquad \nabla S(\boldsymbol{\theta})^\mathsf{T}\mathbf{u} \ge -\|\nabla S(\boldsymbol{\theta})\|.$$

No unit direction can decrease S at a rate faster than $\|\nabla S(\boldsymbol{\theta})\|$.

Step 6: the bound is attained, by exactly one direction. Suppose $\nabla S(\boldsymbol{\theta}) \ne \mathbf{0}$ and put

$$\mathbf{u}^* = -\frac{\nabla S(\boldsymbol{\theta})}{\|\nabla S(\boldsymbol{\theta})\|}.$$

This is a unit vector, and

$$\nabla S(\boldsymbol{\theta})^\mathsf{T}\mathbf{u}^* = -\frac{\nabla S(\boldsymbol{\theta})^\mathsf{T}\nabla S(\boldsymbol{\theta})}{\|\nabla S(\boldsymbol{\theta})\|} = -\frac{\|\nabla S(\boldsymbol{\theta})\|^2}{\|\nabla S(\boldsymbol{\theta})\|} = -\|\nabla S(\boldsymbol{\theta})\|,$$

so the lower bound of Step 5 is achieved. By the equality condition in Step 4, any unit vector achieving it must be a multiple of $\nabla S(\boldsymbol{\theta})$, and the multiple must be negative for the inner product to be negative; so $\mathbf{u}^*$ is the only one.

Conclusion. The direction of fastest decrease is $-\nabla S(\boldsymbol{\theta})$, the *direction of steepest descent*. Geometrically, the gradient is perpendicular to the contour of S through $\boldsymbol{\theta}$, and steepest descent heads straight downhill across the contours. Gradient descent takes

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - t_k\,\nabla S(\boldsymbol{\theta}^{(k)}) = \boldsymbol{\theta}^{(k)} + 2t_k\,\mathbf{J}^\mathsf{T}\mathbf{r},$$

using (2). The direction $\mathbf{J}^\mathsf{T}\mathbf{r} = \sum_i r_i\,(\partial f_i/\partial\boldsymbol{\theta})$ adds up, over observations, the residual times the sensitivity of the fitted value to each parameter. With one parameter the update is $\theta^{(k+1)} = \theta^{(k)} + 2t_k\sum_i J_i r_i$.

3.2 The step length

There are three common ways of choosing t_k.

- A **fixed step** $t_k = t$ is simple, but too large a step overshoots and too small a step crawls.

- An **exact line search** takes $t_k = \arg\min_{t>0} S(\boldsymbol{\theta}^{(k)} + t\mathbf{d}^{(k)})$. It is mostly of theoretical interest, because each step needs a one-dimensional minimization.
- **Backtracking** starts from $t = 1$ and halves t until the *Armijo condition*

$$S(\boldsymbol{\theta} + t\mathbf{d}) \le S(\boldsymbol{\theta}) + c\, t\, \nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{d}, \qquad c = 10^{-4},$$

holds (Armijo, 1966; Nocedal and Wright, 2006). Since $\nabla S^{\mathsf{T}}\mathbf{d} < 0$, this asks for a decrease that is at least a small fraction of what the linear approximation promises. By Lemma 1 some halving always succeeds.

3.3 Example 1: the one-parameter model

We run gradient descent from $\theta^{(0)} = 1.5$ with a small fixed step, a large fixed step, and backtracking.

```
## gradient descent on the one-parameter example, from theta = 1.5
gd1 <- function(th, t = NULL, n = 6) {        # t = NULL: backtracking
  out <- c(k = 0, theta = th, S = S(th))
  for (k in 1:n) {
    g <- dS(th)
    step <- if (is.null(t)) {                  # halve until Armijo holds
      s <- 1
      while (S(th - s * g) > S(th) - 1e-4 * s * g^2) s <- s / 2
      s
    } else t
    th <- th - step * g
    out <- rbind(out, c(k, th, S(th)))
  }
  rownames(out) <- NULL
  round(out, 5)
}
gd1(1.5, t = 0.5)    # a small fixed step
gd1(1.5, t = 4)      # a large fixed step
gd1(1.5)             # backtracking
```

```
     k   theta       S
[1,] 0 1.50000 0.32569
[2,] 1 1.36239 0.28562
[3,] 2 1.20875 0.23567
[4,] 3 1.03745 0.17408
[5,] 4 0.85041 0.10308
[6,] 5 0.66244 0.03914
[7,] 6 0.52332 0.01422
     k   theta       S
[1,] 0 1.50000 0.32569
[2,] 1 0.39909 0.03380
[3,] 2 2.06397 0.44816
[4,] 3 1.39907 0.29674
[5,] 4 0.20443 0.26271
[6,] 5 9.26841 0.61987
[7,] 6 9.26788 0.61987
     k   theta       S
[1,] 0 1.50000 0.32569
[2,] 1 1.22477 0.24113
```

```
[3,] 2 0.88572 0.11644
[4,] 3 0.50547 0.01383
[5,] 4 0.50642 0.01383
[6,] 5 0.50604 0.01383
[7,] 6 0.50619 0.01383
```

- With $t = 0.5$ every step goes the right way, but slowly: after six steps θ is still at 0.52. Far from the minimum the slope is small (the plateau of Figure 1), so the steps are small.
- With $t = 4$ the first step overshoots to 0.40, the second to 2.06, and after a few more the iterate lands at $\theta \approx 9.3$, on the plateau. There $S' \approx 0$, so the method *stops moving*, although $S = 0.62$ is far from the minimum. A small gradient does not mean we are close to a minimum.
- Backtracking chooses the step automatically and reaches $S = 0.0138$ (the minimum) in three steps.

3.4 How fast? The condition number

Near a minimum, S looks like a quadratic, so it is worth understanding steepest descent on a quadratic $S(\mathbf{u}) = \frac{1}{2}\mathbf{u}^\mathsf{T}\mathbf{Q}\mathbf{u}$, with $\mathbf{Q}$ symmetric and positive definite. Let $\lambda_{\max}$ and $\lambda_{\min}$ be the largest and smallest eigenvalues of $\mathbf{Q}$. Their ratio

$$\kappa(\mathbf{Q}) = \frac{\lambda_{\max}(\mathbf{Q})}{\lambda_{\min}(\mathbf{Q})} \geq 1$$

is the *condition number* of $\mathbf{Q}$. It has a simple geometric meaning. The contours $\{\mathbf{u} : \frac{1}{2}\mathbf{u}^\mathsf{T}\mathbf{Q}\mathbf{u} = c\}$ are ellipses (ellipsoids, when $p > 2$) whose semi-axes point along the eigenvectors of $\mathbf{Q}$ and have lengths proportional to $1/\sqrt{\lambda}$. So $\sqrt{\kappa}$ is the ratio of the longest to the shortest axis. If $\kappa = 1$ the contours are circles; if κ is large they are long, thin ellipses, and the function falls steeply in some directions and hardly at all in others. The condition number also measures how sensitive the solution of the linear system $\mathbf{Q}\mathbf{u} = \mathbf{b}$ is to small changes in $\mathbf{b}$: relative errors can be magnified by a factor up to κ.

Theorem 1 (Steepest descent on a quadratic)**.** *Steepest descent with exact line search on* $S(\mathbf{u}) = \frac{1}{2}\mathbf{u}^\mathsf{T}\mathbf{Q}\mathbf{u}$ *satisfies*

$$S(\mathbf{u}^{(k+1)}) \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^2 S(\mathbf{u}^{(k)}), \qquad \kappa = \kappa(\mathbf{Q}).$$

A proof is in Nocedal and Wright (2006, Theorem 3.3). The bound per step is 0.11 for $\kappa = 2$ but 0.85 for $\kappa = 25$, and it tends to 1 as $\kappa \to \infty$: each step then achieves almost nothing. The reason is visible in Figure 3. When the contours are elongated, the steepest direction (perpendicular to the contour) points *across* the valley rather than along it, and the iterates zigzag from one side of the valley to the other.

```
## steepest descent with exact line search on S(u) = u'Qu / 2
steepest <- function(Q, u, n = 200) {
  path <- u
```

```r
  for (k in 1:n) {
    g <- drop(Q %*% u)                          # gradient
    t <- sum(g^2) / sum(g * (Q %*% g))          # exact minimiser along -g
    u <- u - t * g
    path <- rbind(path, u)
  }
  path
}
par(mfrow = c(1, 2))
for (kappa in c(2, 25)) {
  Q    <- diag(c(1, kappa))
  path <- steepest(Q, c(10, 1))
  Sval <- apply(path, 1, function(u) sum(u * (Q %*% u)) / 2)
  nstep <- which(Sval <= 1e-6 * Sval[1])[1] - 1
  u1 <- seq(-11, 11, length.out = 200); u2 <- seq(-3, 3, length.out = 200)
  contour(u1, u2, outer(u1, u2, function(a, b) (a^2 + kappa * b^2) / 2),
          nlevels = 12, drawlabels = FALSE, col = "gray60",
          xlab = expression(u[1]), ylab = expression(u[2]),
          main = sprintf("kappa = %d: %d steps to reduce S by 1e6",
                         kappa, nstep))
  lines(path, type = "o", pch = 20, cex = 0.6, col = "firebrick")
}
```

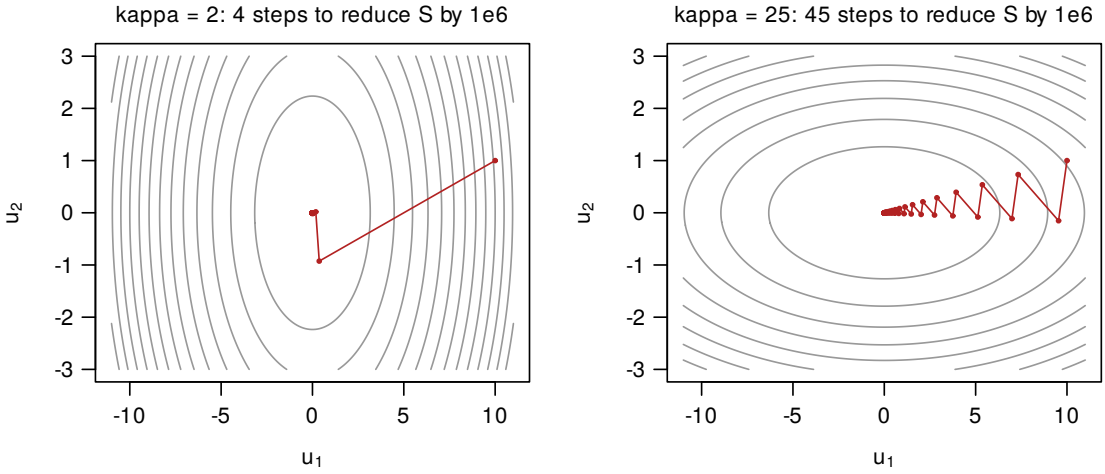


Figure 3: Steepest descent with exact line search on two quadratics $\tfrac{1}{2}\mathbf{u}^{\mathsf{T}}\mathbf{Q}\mathbf{u}$ with $\mathbf{Q} = \mathrm{diag}(1, \kappa)$, started at $(10, 1)$. A condition number of 25 already multiplies the work by more than ten.

3.5 Example 2: Puromycin, and why units matter

For the Michaelis–Menten fit, the relevant quadratic near the minimum has $\mathbf{Q} = \nabla^2 S(\hat{\boldsymbol{\theta}})$. Because $V_m \approx 200$ while $K \approx 0.06$, a unit step in K is enormous and a unit step in V_m is tiny, and the contours of S in the (V_m, K) plane are extremely elongated. The cheapest remedy is to *rescale* the parameters: write $\boldsymbol{\theta} = \mathbf{D}\boldsymbol{\phi}$ with $\mathbf{D} = \mathrm{diag}(100,\, 0.03)$, so that both components of $\boldsymbol{\phi}$ are of order one. In the new coordinates the Hessian is $\mathbf{D}\,\nabla^2 S\,\mathbf{D}$, and gradient descent in $\boldsymbol{\phi}$, written in terms of $\boldsymbol{\theta}$, is

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - t_k\, \mathbf{D}^2\, \nabla S(\boldsymbol{\theta}^{(k)}).$$

```
H  <- hessS(th_hat)                              # Hessian at the minimum
D  <- diag(c(100, 0.03))                         # rescaling theta = D phi
kap <- c(original = kappa(H, exact = TRUE),
         rescaled = kappa(D %*% H %*% D, exact = TRUE))
rbind(condition.number = kap,
      bound.per.step = ((kap - 1) / (kap + 1))^2)
```

```
                     original   rescaled
condition.number 1.731260e+06 45.3914672
bound.per.step   9.999977e-01  0.9156358
```

In the original units the condition number of the Hessian is 1.7×10^6, and the bound of Theorem 1 per step is 0.999998: essentially no progress. After rescaling the condition number is 45 and the bound is 0.92. The practical difference is dramatic.

```
## gradient descent with backtracking; D rescales the parameters
grad_descent <- function(th, D = c(1, 1), nstep = 100) {
  for (k in 1:nstep) {
    g <- gradS(th)
    d <- -D^2 * g                                  # search direction
    t <- 1
    while (S(th + t * d) > S(th) + 1e-4 * t * sum(g * d)) t <- t / 2
    th <- unname(th + t * d)
  }
  c(Vm = th[1], K = th[2], S = S(th))
}
rbind(unscaled.20000.steps = grad_descent(A, nstep = 20000),
      rescaled.100.steps   = grad_descent(A, D = c(100, 0.03), nstep = 100),
      minimum              = c(th_hat, S(th_hat)))
```

```
                          Vm          K        S
unscaled.20000.steps 248.6559 0.10472336 3728.287
rescaled.100.steps   212.5815 0.06393805 1195.503
minimum              212.6837 0.06412128 1195.449
```

After 20 000 steps from A, unscaled gradient descent has reached (248.7, 0.105) with $S = 3728$; the minimum is $S = 1195.4$. The first few steps fix K roughly, and after that the method barely moves in V_m (the orange path in Figure 5). With rescaling, 100 steps reach $S = 1195.50$.

When is gradient descent the right tool? It needs only ∇S, so each step is cheap even when p is in the millions, and with a line search it never increases S. Its weaknesses are a merely linear rate of convergence, governed by the condition number, and sensitivity to the units of the parameters. Classical non-linear regression has few parameters and a cheap Jacobian; there, methods that also use curvature are much better.

4 Newton's method

4.1 The method

Newton's method replaces S near the current iterate by its second-order Taylor approximation,

$$S(\boldsymbol{\theta}+\mathbf{d}) \approx m(\mathbf{d}) = S(\boldsymbol{\theta}) + \nabla S(\boldsymbol{\theta})^{\mathsf{T}}\mathbf{d} + \tfrac{1}{2}\,\mathbf{d}^{\mathsf{T}}\mathbf{H}\,\mathbf{d}, \qquad \mathbf{H} = \nabla^2 S(\boldsymbol{\theta}),$$

and moves to the minimizer of this quadratic model. Setting $\nabla m(\mathbf{d}) = \nabla S(\boldsymbol{\theta}) + \mathbf{H}\mathbf{d} = \mathbf{0}$ gives the *Newton step*

$$\mathbf{d}_N = -\mathbf{H}^{-1}\nabla S(\boldsymbol{\theta}), \qquad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \mathbf{d}_N.$$

The stationary point of m is a *minimum* only if $\mathbf{H}$ is positive definite. In that case $\mathbf{d}_N$ is a descent direction, because $\nabla S^{\mathsf{T}}\mathbf{d}_N = -\nabla S^{\mathsf{T}}\mathbf{H}^{-1}\nabla S < 0$.

For least squares, $\mathbf{H}$ is given by (3). With one parameter, (4) gives

$$\theta^{(k+1)} = \theta^{(k)} - \frac{S'(\theta^{(k)})}{S''(\theta^{(k)})} = \theta^{(k)} + \frac{\sum_i J_i r_i}{\sum_i J_i^2 - \sum_i r_i f_i''}.$$

This is the Newton–Raphson method for solving the equation $S'(\theta) = 0$: draw the tangent to the curve S' at the current point (right panel of Figure 1) and move to where the tangent crosses zero. Newton's method does not depend on the units of the parameters: rescaling $\boldsymbol{\theta} = \mathbf{D}\boldsymbol{\phi}$ gives exactly the same iterates.

Theorem 2 (Local quadratic convergence)**.** *Suppose $\nabla^2 S$ is Lipschitz continuous near $\widehat{\boldsymbol{\theta}}$, that $\nabla S(\widehat{\boldsymbol{\theta}}) = \mathbf{0}$ and that $\nabla^2 S(\widehat{\boldsymbol{\theta}})$ is positive definite. If $\boldsymbol{\theta}^{(0)}$ is close enough to $\widehat{\boldsymbol{\theta}}$, the Newton iterates converge to $\widehat{\boldsymbol{\theta}}$, and for some constant C,*

$$\|\boldsymbol{\theta}^{(k+1)} - \widehat{\boldsymbol{\theta}}\| \le C\,\|\boldsymbol{\theta}^{(k)} - \widehat{\boldsymbol{\theta}}\|^2.$$

Idea of the proof. Write $\mathbf{e}_k = \boldsymbol{\theta}^{(k)} - \widehat{\boldsymbol{\theta}}$ and $\mathbf{H} = \nabla^2 S(\boldsymbol{\theta}^{(k)})$. Expanding the gradient around $\boldsymbol{\theta}^{(k)}$ and using $\nabla S(\widehat{\boldsymbol{\theta}}) = \mathbf{0}$ gives $\nabla S(\boldsymbol{\theta}^{(k)}) = \mathbf{H}\mathbf{e}_k + O(\|\mathbf{e}_k\|^2)$. Hence $\mathbf{e}_{k+1} = \mathbf{e}_k - \mathbf{H}^{-1}\nabla S(\boldsymbol{\theta}^{(k)}) = O(\|\mathbf{e}_k\|^2)$. The details are in Nocedal and Wright (2006, Theorem 3.5). □

Quadratic convergence means that, once we are close, the number of correct digits roughly *doubles* at every step. The theorem is *local*: it says nothing about what happens far from $\widehat{\boldsymbol{\theta}}$.

4.2 Example 1: the one-parameter model

```
newton1 <- function(th, n = 5) {
  out <- c(k = 0, theta = th, error = th - th_hat, S2 = d2S(th))
  for (k in 1:n) {
    th <- th - dS(th) / d2S(th)                 # Newton step
    out <- rbind(out, c(k, th, th - th_hat, d2S(th)))
  }
```

```
  rownames(out) <- NULL
  out
}
signif(newton1(0.3), 4)                          # start close to the minimum
signif(newton1(1.5, n = 3), 4)                   # start where S'' < 0
```

```
     k  theta      error    S2
[1,] 0 0.3000 -2.061e-01 9.147
[2,] 1 0.4214 -8.473e-02 4.591
[3,] 2 0.4883 -1.789e-02 3.104
[4,] 3 0.5052 -9.222e-04 2.805
[5,] 4 0.5061 -2.549e-06 2.789
[6,] 5 0.5061  2.090e-09 2.789
     k theta  error       S2
[1,] 0 1.500 0.9939 -0.22860
[2,] 1 2.704 2.1980 -0.08731
[3,] 2 3.740 3.2330 -0.03241
[4,] 3 4.753 4.2460 -0.01196
```

From $\theta^{(0)} = 0.3$ the errors are 0.21, 0.085, 0.018, 9×10^{-4}, 3×10^{-6}, 2×10^{-9}: after the first two steps the exponent roughly doubles each time, as Theorem 2 promises. From $\theta^{(0)} = 1.5$ things go wrong. There $S' > 0$ (we should move left) but $S'' < 0$ (the shaded region of Figure 1), so the Newton step $-S'/S''$ is *positive*: Newton moves to the right, towards the stationary point of a quadratic model that opens downwards, and keeps going out along the plateau. Newton's method looks for a zero of S'; it does not know whether that zero is a minimum.

4.3 Example 2: Puromycin

```
newton_step <- function(th) -solve(hessS(th), gradS(th))
iterate <- function(step, th, n = 7) {           # n full steps from th
  path <- th
  for (k in 1:n) { th <- th + step(th); path <- rbind(path, th) }
  unname(path)
}
relerr <- function(path)                         # largest relative error
  apply(path, 1, function(th) max(abs(th - th_hat) / abs(th_hat)))
pN <- iterate(newton_step, A)
data.frame(k = 0:7, Vm = round(pN[, 1], 3), K = round(pN[, 2], 5),
           rel.error = signif(relerr(pN), 2))
```

```
  k      Vm       K rel.error
1 0 250.000 0.03000   5.3e-01
2 1 184.495 0.02880   5.5e-01
3 2 200.879 0.04490   3.0e-01
4 3 209.110 0.05749   1.0e-01
5 4 212.236 0.06323   1.4e-02
6 5 212.675 0.06410   2.8e-04
7 6 212.684 0.06412   1.1e-07
8 7 212.684 0.06412   1.7e-14
```

From A the first step actually makes things slightly worse (K goes from 0.030 to 0.029), but once the iterates are close the relative error falls as $10^{-2} \to 10^{-4} \to 10^{-7} \to 10^{-14}$.

Now start from B.

```
iterate(newton_step, B, 4)                    # full Newton steps from B
## the Hessian at B is not positive definite
eigen(hessS(B))$values
```

```
          [,1]      [,2]
[1,]  150.0000 0.2000000
[2,]  272.8445 0.5429167
[3,]  439.6450 1.1349094
[4,]  682.4890 2.0798549
[5,] 1045.7708 3.5382583
[1]  6.886227e+00 -7.126042e+05
```

The Newton iterates march off with both parameters growing without bound. The reason is in the last line: at B the Hessian has one positive and one large negative eigenvalue, so it is not positive definite, the quadratic model has a saddle rather than a minimum, and the Newton step need not be a descent direction.

Remedies. Add a line search (damped Newton); modify the Hessian to make it positive definite, for example by adding a multiple of the identity; or use a *trust region*, minimizing the quadratic model only within a ball where it can be trusted (Dennis and Schnabel, 1996; Nocedal and Wright, 2006). For least squares there is a cleverer modification: drop the troublesome part of the Hessian.

5 The Gauss–Newton method

5.1 Two derivations

Derivation 1: approximate the Hessian. Drop the second term in (3), so that $\nabla^2 S \approx 2\mathbf{J}^\mathsf{T}\mathbf{J}$. The Newton step becomes

$$\mathbf{d}_{GN} = -(2\mathbf{J}^\mathsf{T}\mathbf{J})^{-1}\,(-2\mathbf{J}^\mathsf{T}\mathbf{r}) = (\mathbf{J}^\mathsf{T}\mathbf{J})^{-1}\mathbf{J}^\mathsf{T}\mathbf{r}. \tag{5}$$

Derivation 2: linearize the model. Near the current iterate, a first-order Taylor expansion gives

$$\mathbf{f}(\boldsymbol{\theta}) \approx \mathbf{f}(\boldsymbol{\theta}^{(k)}) + \mathbf{J}\,(\boldsymbol{\theta} - \boldsymbol{\theta}^{(k)}), \qquad \mathbf{J} = \mathbf{J}(\boldsymbol{\theta}^{(k)}).$$

Substituting into S and writing $\boldsymbol{\delta} = \boldsymbol{\theta} - \boldsymbol{\theta}^{(k)}$ and $\mathbf{r}^{(k)} = \mathbf{y} - \mathbf{f}(\boldsymbol{\theta}^{(k)})$,

$$S(\boldsymbol{\theta}) \approx \|\mathbf{y} - \mathbf{f}(\boldsymbol{\theta}^{(k)}) - \mathbf{J}\boldsymbol{\delta}\|^2 = \|\mathbf{r}^{(k)} - \mathbf{J}\boldsymbol{\delta}\|^2.$$

This is the sum of squares of an *ordinary linear regression* with "response" $\mathbf{r}^{(k)}$ and "design matrix" $\mathbf{J}$. Its least-squares solution is $\boldsymbol{\delta} = (\mathbf{J}^\mathsf{T}\mathbf{J})^{-1}\mathbf{J}^\mathsf{T}\mathbf{r}^{(k)}$: the same step as (5).

So each Gauss–Newton iteration *is* a linear regression: regress the current residuals on the current Jacobian (without an intercept), and add the coefficients to the current estimate. With one parameter,

$$\theta^{(k+1)} = \theta^{(k)} + \frac{\sum_i J_i r_i}{\sum_i J_i^2}, \tag{6}$$

the least-squares slope of the residuals on the sensitivities. Compared with Newton's step, the term $-\sum_i r_i f_i''$ has disappeared from the denominator. Since $\sum_i J_i^2 > 0$ always, the step always points downhill.

Proposition 1 (Gauss–Newton goes downhill)**.** *If* $\mathbf{J}$ *has full column rank and* $\mathbf{J}^\top\mathbf{r} \neq \mathbf{0}$*, then* $\mathbf{d}_{GN}$ *is a descent direction.*

Proof. By (2) and (5),

$$\nabla S^\top \mathbf{d}_{GN} = -2\,\mathbf{r}^\top\mathbf{J}(\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{J}^\top\mathbf{r} = -2\,\|\mathbf{P}_\mathbf{J}\mathbf{r}\|^2 < 0,$$

where $\mathbf{P}_\mathbf{J} = \mathbf{J}(\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{J}^\top$ is the orthogonal projection onto the column space of $\mathbf{J}$ (we used $\mathbf{P}_\mathbf{J} = \mathbf{P}_\mathbf{J}^\top\mathbf{P}_\mathbf{J}$). The projection $\mathbf{P}_\mathbf{J}\mathbf{r}$ is non-zero because $\mathbf{J}^\top\mathbf{r} = \mathbf{J}^\top\mathbf{P}_\mathbf{J}\mathbf{r} \neq \mathbf{0}$. □

Unlike Newton's method there are no worries about positive definiteness: $\mathbf{J}^\top\mathbf{J}$ is always positive semi-definite, and positive definite when $\mathbf{J}$ has full column rank. But "descent direction" only guarantees improvement for a *small enough* step (Lemma 1); a full step can still overshoot.

5.2 Example 1: the one-parameter model

```
gn1 <- function(th, n = 6) {
  out <- c(k = 0, theta = th, error = th - th_hat)
  for (k in 1:n) {
    th <- th + sum(J(th) * (y - f(th))) / sum(J(th)^2)  # Gauss-Newton step
    out <- rbind(out, c(k, th, th - th_hat))
  }
  rownames(out) <- NULL
  cbind(signif(out, 4), ratio = c(NA, signif(out[-1, 3] / out[-nrow(out), 3], 3)))
}
gn1(1, n = 7)                                          # start at theta = 1
## the predicted ratio: sum(f'' r) / sum(J^2) at the minimum
sum(f2(th_hat) * (y - f(th_hat))) / sum(J(th_hat)^2)
```

```
     k  theta      error   ratio
[1,] 0 1.0000  4.939e-01      NA
[2,] 1 0.1807 -3.255e-01 -0.6590
[3,] 2 0.4149 -9.124e-02  0.2800
[4,] 3 0.5017 -4.470e-03  0.0490
[5,] 4 0.5064  2.171e-04 -0.0486
[6,] 5 0.5061 -1.172e-05 -0.0540
[7,] 6 0.5061  6.315e-07 -0.0539
[8,] 7 0.5061 -3.171e-08 -0.0502
[1] -0.05372856
```

The first step from $\theta^{(0)} = 1$ overshoots to 0.18, on the other side of the minimum; after that the iterates close in quickly, and the error changes sign at every step from $k = 3$ on. The last column is the ratio of successive errors, which settles at about -0.054. Where does that number come from? Write (6) as $\theta^{(k+1)} = G(\theta^{(k)})$ with $G(\theta) = \theta + N(\theta)/D(\theta)$, $N = \sum_i J_i r_i$ and $D = \sum_i J_i^2$. At the minimum $N(\widehat{\theta}) = 0$ (the normal equation), so

$$G'(\widehat{\theta}) = 1 + \frac{N'(\widehat{\theta})}{D(\widehat{\theta})}, \qquad N' = \sum_i \left(f_i'' r_i + J_i r_i'\right) = \sum_i \left(f_i'' r_i - J_i^2\right),$$

because $r_i' = -J_i$. Hence

$$G'(\widehat{\theta}) = \frac{\sum_i f_i''(\widehat{\theta})\, r_i(\widehat{\theta})}{\sum_i J_i(\widehat{\theta})^2}.$$

Near the minimum $e_{k+1} \approx G'(\widehat{\theta})\, e_k$, where $e_k = \theta^{(k)} - \widehat{\theta}$: convergence is *linear* with factor $|G'(\widehat{\theta})| = 0.054$, and because $G'(\widehat{\theta}) < 0$ the iterates alternate around $\widehat{\theta}$. The last line of the output computes $G'(\widehat{\theta})$ directly. Two lessons follow. The rate is fast when the residuals are small or the model is nearly linear ($f'' \approx 0$), exactly the two situations in which the dropped term of (3) is small. And if $|G'(\widehat{\theta})| > 1$, the full-step method would fail to converge even when started very close to $\widehat{\theta}$.

5.3 Example 2: Puromycin, one step as a regression

Here is a single Gauss–Newton step from A, computed literally as a linear regression of the residuals on the two columns of the Jacobian.

```
## one Gauss-Newton step from A, done as a linear regression
r0  <- y - f(A)                                  # current residuals
J0  <- Jac(A)                                    # current Jacobian
reg <- lm(r0 ~ dVm + dK - 1,                     # regress r on J, no intercept
          data = data.frame(r0, dVm = J0[, 1], dK = J0[, 2]))
coef(reg)                                        # the step (delta Vm, delta K)
A + coef(reg)                                    # the new iterate
c(S.before = S(A), S.after = S(A + coef(reg)))
```

```
        dVm          dK
-44.3867154   0.0164387
        dVm          dK
205.6132846   0.0464387
 S.before   S.after
35495.362  2013.275
```

The regression says: decrease V_m by 44.4 and increase K by 0.016. That matches what we saw in Figure 2. At A the curve is too high at large concentrations, a pattern that matches the V_m column of $\mathbf{J}$ (largest at high x), so V_m comes down; and the curve rises too steeply at low concentrations, which a larger K corrects. One step reduces S from 35 495 to 2013, by a factor of 18. Continuing:

```
gn_step <- function(th) {                        # least squares of r on J
  J <- Jac(th)
  drop(solve(crossprod(J), crossprod(J, y - f(th))))
}
## relative offset of Bates and Watts (n = 12, p = 2)
offset <- function(th) {
  qtr <- qr.qty(qr(Jac(th)), y - f(th))          # Q'r: first p entries are Q1'r
  sqrt(sum(qtr[1:2]^2) / 2) / sqrt(sum(qtr[-(1:2)]^2) / 10)
}
path <- iterate(gn_step, A)
data.frame(k = 0:7, Vm = round(path[, 1], 3), K = round(path[, 2], 5),
           S = round(apply(path, 1, S), 4),
           offset = signif(apply(path, 1, offset), 2))
```

```
  k      Vm       K         S  offset
```

```
1 0 250.000 0.03000 35495.363 1.0e+01
2 1 205.613 0.04644  2013.275 1.7e+00
3 2 210.524 0.05997  1226.002 3.4e-01
4 3 212.376 0.06360  1195.876 4.0e-02
5 4 212.651 0.06407  1195.453 4.0e-03
6 5 212.681 0.06412  1195.449 3.8e-04
7 6 212.683 0.06412  1195.449 3.7e-05
8 7 212.684 0.06412  1195.449 3.6e-06
```

After four steps S is correct to six significant digits. The last column, the *relative offset*, is a stopping criterion explained in Section 7; it falls by a factor of about 10 per step.

5.4 How fast is Gauss–Newton?

The one-parameter calculation generalizes. Gauss–Newton is Newton's method with the Hessian error $2\mathbf{E}$, $\mathbf{E} = \sum_i \widehat{r}_i \nabla^2 f_i(\widehat{\boldsymbol{\theta}})$, and linearizing the iteration around $\widehat{\boldsymbol{\theta}}$ gives

$$\mathbf{e}_{k+1} \approx (\mathbf{J}^{\mathsf{T}}\mathbf{J})^{-1}\mathbf{E}\,\mathbf{e}_k,$$

with everything evaluated at $\widehat{\boldsymbol{\theta}}$. The convergence is linear, with factor ρ, the spectral radius (largest absolute eigenvalue) of $(\mathbf{J}^{\mathsf{T}}\mathbf{J})^{-1}\mathbf{E}$. If the residuals at the solution are exactly zero then $\mathbf{E} = \mathbf{0}$ and the convergence is quadratic (Dennis and Schnabel, 1996).

```
data.frame(k = 0:7,
           Newton       = signif(relerr(iterate(newton_step, A)), 2),
           Gauss.Newton = signif(relerr(iterate(gn_step, A)), 2))
## predicted linear rate: spectral radius of (J'J)^{-1} E at the minimum
JtJ <- crossprod(Jac(th_hat))
max(abs(eigen(solve(JtJ, JtJ - hessS(th_hat) / 2))$values))
```

```
  k  Newton Gauss.Newton
1 0 5.3e-01      5.3e-01
2 1 5.5e-01      2.8e-01
3 2 3.0e-01      6.5e-02
4 3 1.0e-01      8.1e-03
5 4 1.4e-02      8.0e-04
6 5 2.8e-04      7.8e-05
7 6 1.1e-07      7.5e-06
8 7 1.7e-14      7.2e-07
[1] 0.09635013
```

Newton is slow at first and then quadratic. Gauss–Newton is faster at first and then gains about one digit per step, linearly with factor $\rho = 0.096$. In practice the Gauss–Newton method has two advantages that outweigh its slower final rate: it needs only first derivatives, and it never needs to worry about an indefinite Hessian.

5.5 Example 3: when the Jacobian loses a column

The biochemical oxygen demand (BOD) data, built into R as `BOD`, have six observations of the oxygen demand of a water sample after $t = 1, 2, 3, 4, 5, 7$ days, and are fitted by

$$f(t;\boldsymbol{\theta}) = \theta_1\bigl(1 - e^{-\theta_2 t}\bigr), \qquad \frac{\partial f}{\partial \theta_1} = 1 - e^{-\theta_2 t}, \qquad \frac{\partial f}{\partial \theta_2} = \theta_1\, t\, e^{-\theta_2 t}.$$

The least-squares estimate is $\widehat{\boldsymbol{\theta}} = (19.14,\ 0.531)$. Compare the Jacobian at a reasonable start, (20, 0.5), with the Jacobian at (20, 5).

```
tb <- BOD$Time; yb <- BOD$demand
Jbod <- function(th) cbind(th1 = 1 - exp(-th[2] * tb),        # df/dth1
                           th2 = th[1] * tb * exp(-th[2] * tb)) # df/dth2
round(cbind(Time = tb, Jbod(c(20, 0.5)), Jbod(c(20, 5))), 4)
## one Gauss-Newton step from (20, 5)
th0 <- c(20, 5); Jb <- Jbod(th0); rb <- yb - th0[1] * (1 - exp(-th0[2] * tb))
th0 + drop(solve(crossprod(Jb), crossprod(Jb, rb)))
```

```
     Time    th1     th2    th1    th2
[1,]    1 0.3935 12.1306 0.9933 0.1348
[2,]    2 0.6321 14.7152 1.0000 0.0018
[3,]    3 0.7769 13.3878 1.0000 0.0000
[4,]    4 0.8647 10.8268 1.0000 0.0000
[5,]    5 0.9179  8.2085 1.0000 0.0000
[6,]    7 0.9698  4.2276 1.0000 0.0000
      th1       th2
 16.17693 -53.21736
```

At $\theta_2 = 5$ the curve has already reached its asymptote by $t = 1$, so the fitted values hardly depend on θ_2: the second column of $\mathbf{J}$ is essentially zero except in the first row. The regression of the residuals on $\mathbf{J}$ then has an almost useless second regressor, its coefficient is wildly unstable, and the step sends θ_2 to -53. With θ_2 negative, $e^{-\theta_2 t}$ explodes; this is the origin of the error messages "singular gradient" and "Missing value or an infinity produced when evaluating the model" that `nls` gives from such starts.

```
tg <- seq(0, 8, length.out = 300)
plot(tg, 20 * tg * exp(-0.5 * tg), type = "l", lwd = 2, col = "steelblue",
     xlab = "time (days)", ylab = expression(partialdiff * f / partialdiff * theta
    [2]))
lines(tg, 20 * tg * exp(-5 * tg), lwd = 2, lty = 2, col = "firebrick")
rug(tb, lwd = 2)
legend("topright", bty = "n", lwd = 2, lty = 1:2,
       col = c("steelblue", "firebrick"),
       c(expression(theta == (20 * ", " * 0.5)), expression(theta == (20 * ", " * 5)
    )))
```

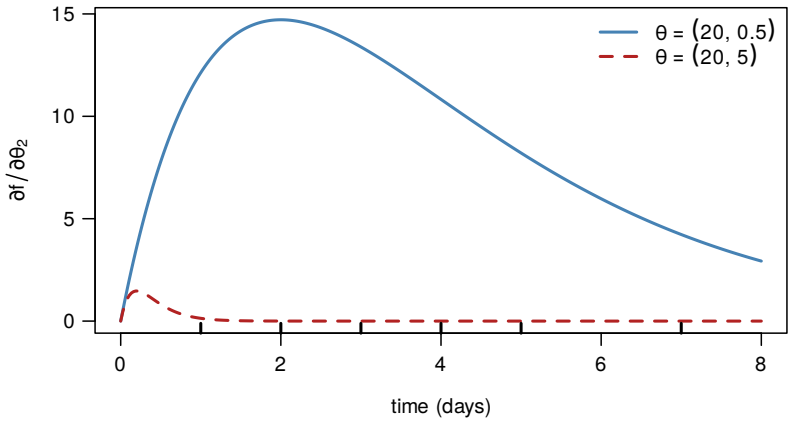


Figure 4: The sensitivity $\partial f/\partial\theta_2$ of the BOD model as a function of time, at two parameter values. The tick marks on the axis are the observation times. At $\theta_2 = 5$ the sensitivity is negligible at every observation time except the first.

Remark 1. Gauss–Newton is only as good as the linear regression it solves at each step. When two columns of $\mathbf{J}$ are nearly proportional, or one is nearly zero, the regression is ill-conditioned, and so is the step. Watching the columns of the Jacobian is the quickest way to understand a failing fit.

5.6 Making Gauss–Newton safe: step halving

What happens from the other Puromycin starting point?

```
iterate(gn_step, B, 2)                              # full Gauss-Newton steps from B
## Gauss-Newton with step halving (Hartley)
gn_halving <- function(th, tol = 1e-5, maxit = 50) {
  path <- th
  while (offset(th) > tol && nrow(rbind(path)) <= maxit) {
    d <- gn_step(th); t <- 1
    while (S(th + t * d) >= S(th)) t <- t / 2     # halve until S decreases
    th <- th + t * d
    path <- rbind(path, th)
  }
  unname(path)
}
pB <- gn_halving(B)
round(head(pB, 4), 4)                               # the first three steps
c(iterations = nrow(pB) - 1, Vm = pB[nrow(pB), 1], K = pB[nrow(pB), 2])
```

```
          [,1]       [,2]
[1,] 150.00000  0.2000000
[2,] 192.17161 -0.1713994
[3,]  12.01715 -0.1797498
         [,1]   [,2]
[1,] 150.0000 0.2000
[2,] 171.0858 0.0143
[3,] 195.6361 0.0375
[4,] 208.3381 0.0563
  iterations           Vm            K
```

```
8.00000000 212.68367271   0.06412117
```

From B the first full Gauss–Newton step lands at $K = -0.17$, a *negative* half-saturation constant, and the second goes to $V_m = 12$: the method diverges. Hartley (1961) proposed a simple safeguard, the *modified Gauss–Newton method*:

1. compute $\mathbf{d}_{GN}$ by least squares of $\mathbf{r}$ on $\mathbf{J}$;
2. try $t = 1, \frac{1}{2}, \frac{1}{4}, \ldots$ until $S(\boldsymbol{\theta} + t\,\mathbf{d}_{GN}) < S(\boldsymbol{\theta})$;
3. update and repeat.

Because $\mathbf{d}_{GN}$ is a descent direction (Proposition 1), step 2 always succeeds after finitely many halvings (Lemma 1). From B the first accepted step is a much shorter one, to (171.1, 0.014), which keeps K positive, and the iteration converges in 8 steps. This is exactly what R's `nls` does by default: the step factor is halved until S decreases, and if it falls below `minFactor` $= 1/1024$ the function stops with the error "step factor reduced below minFactor".

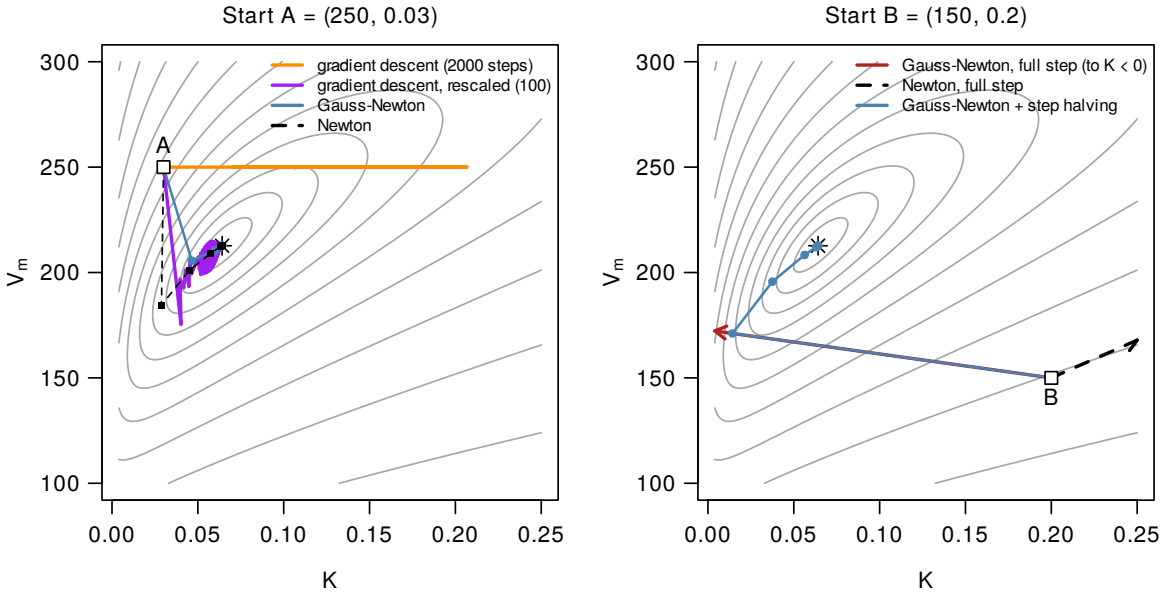


Figure 5: Paths on the Puromycin sum-of-squares surface (contours of $\log_{10} S$; the star is the minimum). Left, from A: Gauss–Newton and Newton reach the minimum in a few steps; unscaled gradient descent fixes K and then hardly moves in V_m; rescaled gradient descent gets there slowly. Right, from B: the full Gauss–Newton step leaves the region $K > 0$ and the full Newton step heads the wrong way (both arrows are clipped at the edge of the plot), while Gauss–Newton with step halving converges (code: `code/n4_paths_fig.R`).

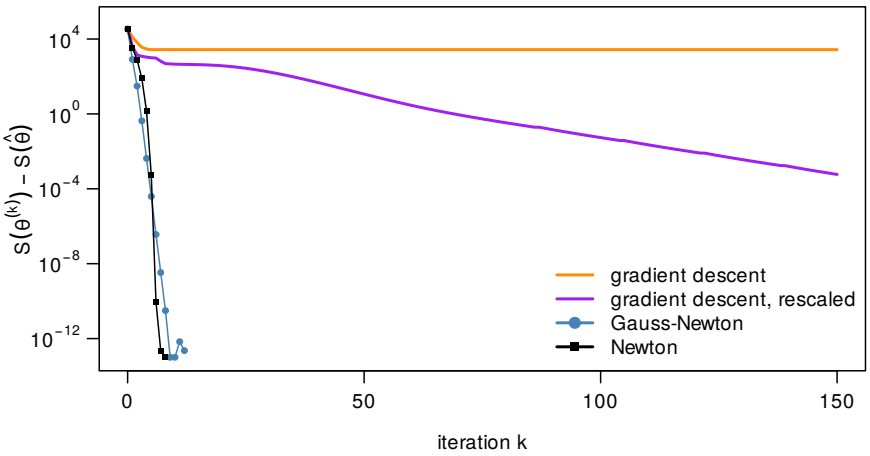


Figure 6: Distance from the minimum, $S(\boldsymbol{\theta}^{(k)}) - S(\widehat{\boldsymbol{\theta}})$, against the iteration number, from A (code: `code/n4_conv_fig.R`).

6 Starting values and self-starting models

6.1 Why starting values matter

Every result in this handout about convergence is *local*: Newton's method converges quadratically *if* it starts close enough (Theorem 2), and the Gauss–Newton rate is a statement about what happens near $\widehat{\boldsymbol{\theta}}$. The Puromycin start B and the BOD start $(20, 5)$ show what a poor start can do: divergence, a negative half-saturation constant, an overflowing exponential. When the sum-of-squares surface has several local minima, the start also decides *which* minimum we find. Good starting values are not a detail.

6.2 Five ways to find starting values

1. **Use the meaning of the parameters.** In the Michaelis–Menten model V_m is the asymptote, so start at the largest observed rate, $V_m^{(0)} = 207$. K is the concentration at half the maximum rate; rates near $207/2 \approx 104$ occur at about $x = 0.06$, so $K^{(0)} = 0.06$.

2. **Use a linearization.** Fit a transformed model by ordinary least squares and transform back. Taking reciprocals of the Michaelis–Menten model gives the straight line $1/f = 1/V_m + (K/V_m)(1/x)$ (Lineweaver and Burk, 1934); regressing $1/y$ on $1/x$ gives $(195.8,\ 0.048)$. This is a poor *estimate*, because the reciprocal transformation distorts the errors, but an excellent *start*.

3. **Look at pictures.** Plot the data and overlay the curve for a few trial values of the parameters, changing one parameter at a time until the curve is close to the data (Ritz and Streibig, 2008, Section 3.1).

4. **Search a grid.** Evaluate S on a coarse grid of parameter values and start from

the best grid point. When some parameters enter linearly, only the others need a grid (Section 6.4).

5. **Use a self-starting model** (Section 6.3), which packages one of the strategies above so that it is applied automatically.

Whatever the strategy, it is wise to try several starts. If they lead to different answers, that is a diagnostic to be reported, not a nuisance to be hidden.

```
n_iter <- function(start) nrow(gn_halving(start)) - 1
c(meaning = n_iter(c(207, 0.06)), lineweaver.burk = n_iter(c(195.8, 0.0484)),
  A = n_iter(A), B = n_iter(B))                  # iterations from four starts
```

```
        meaning lineweaver.burk               A               B
              5               6               7               8
```

The iteration counts are for Gauss–Newton with step halving, stopped when the relative offset falls below 10^{-5}. Good starts save a few iterations, but their real value is that they keep the algorithm out of trouble. A grid search takes a few lines.

```
grid <- expand.grid(Vm = seq(100, 300, by = 25), K = c(0.01, 0.02, 0.05, 0.1, 0.2, 0
    .5))
grid$S <- apply(grid[, c("Vm", "K")], 1, S)       # S at each of 54 grid points
best <- grid[which.min(grid$S), ]
best
fit_grid <- nls(rate ~ Vm * conc / (K + conc), data = pur,
                start = list(Vm = best$Vm, K = best$K))
coef(fit_grid)
```

```
    Vm    K        S
23 200 0.05 1636.586
          Vm            K
212.68357997   0.06412103
```

6.3 Self-starting models

For a model that is fitted again and again (a growth curve fitted to every field trial, a pharmacokinetic model fitted to every patient) one does not want to hunt for starting values each time. A *self-starting model*, or *self-starter*, is a mean function packaged together with a routine that computes starting values for it *from the data* (Pinheiro and Bates, 2000; Ritz and Streibig, 2008). Once a self-starter exists, the model can be fitted without supplying any starting values at all.

What a self-starter consists of. In R a self-starter is an object created by the function `selfStart()`, from three ingredients:

1. **the mean function**, an ordinary R function whose first argument is the explanatory variable and whose remaining arguments are the parameters, for example `function(x, Vm, K) Vm * x / (K + x)`; it may also return the Jacobian as a `"gradient"` attribute, so that `nls` need not use finite differences;

2. **an initial-value routine**, a function with arguments `(mCall, data, LHS, ...)` that receives the data from `nls` and returns a named vector of starting values;

3. **the names of the parameters**.

When `nls` is called with a self-starter in the model formula and *without* the argument `start`, it calls `getInitial()`, which runs the initial-value routine on the data and uses the result as $\boldsymbol{\theta}^{(0)}$. If `start` is supplied it overrides the routine.

The built-in self-starters. R comes with ten of them.

Function	Mean function	Model
`SSasymp(x, Asym, R0, lrc)`	$\text{Asym} + (\text{R0} - \text{Asym})\, e^{-e^{\text{lrc}}x}$	asymptotic regression
`SSasympOff(x, Asym, lrc, c0)`	$\text{Asym}\,\{1 - e^{-e^{\text{lrc}}(x - c_0)}\}$	with an offset
`SSasympOrig(x, Asym, lrc)`	$\text{Asym}\,(1 - e^{-e^{\text{lrc}}x})$	through the origin
`SSbiexp(x, A1, lrc1, A2, lrc2)`	$A_1 e^{-e^{\text{lrc1}}x} + A_2 e^{-e^{\text{lrc2}}x}$	two exponentials
`SSfol(Dose, t, lKe, lKa, lCl)`	$\dfrac{D\, e^{\text{lKe}+\text{lKa}-\text{lCl}}\,(e^{-e^{\text{lKe}}t} - e^{-e^{\text{lKa}}t})}{e^{\text{lKa}} - e^{\text{lKe}}}$	one compartment, oral
`SSfpl(x, A, B, xmid, scal)`	$A + \dfrac{B - A}{1 + e^{(\text{xmid} - x)/\text{scal}}}$	four-parameter logistic
`SSgompertz(x, Asym, b2, b3)`	$\text{Asym}\, e^{-b_2 b_3^{x}}$	Gompertz
`SSlogis(x, Asym, xmid, scal)`	$\dfrac{\text{Asym}}{1 + e^{(\text{xmid} - x)/\text{scal}}}$	logistic
`SSmicmen(x, Vm, K)`	$\dfrac{V_m\, x}{K + x}$	Michaelis–Menten
`SSweibull(x, Asym, Drop, lrc, pwr)`	$\text{Asym} - \text{Drop}\, e^{-e^{\text{lrc}}x^{\text{pwr}}}$	Weibull growth

Notice that rate constants enter through their logarithms (`lrc`, `lKe`, `lKa`, `lCl`). That keeps them positive whatever the algorithm does, a cheap guard against the kind of failure we saw from start B.

```
apropos("^SS", where = FALSE)[-1]                # built-in self-starters
fit_ss <- nls(rate ~ SSmicmen(conc, Vm, K), data = pur)   # no start needed
coef(fit_ss)
getInitial(rate ~ SSmicmen(conc, Vm, K), data = pur)     # its starting values
fit_ss$convInfo$finIter                          # iterations used by nls
```

```
 [1] "SSasymp"     "SSasympOff"  "SSasympOrig" "SSbiexp"     "SSfol"
 [6] "SSfpl"       "SSgompertz"  "SSlogis"     "SSmicmen"    "SSweibull"
          Vm             K
212.68370727   0.06412123
          Vm             K
212.68370727   0.06412123
[1] 0
```

No starting values were given, yet `nls` found the estimate; and the number of iterations it needed after the self-starter was *zero*. The "starting values" returned by `getInitial` are already the least-squares estimates.

Inside `SSmicmen`. Why zero iterations? Here is the initial-value routine of `SSmicmen`, and the same three steps carried out by hand.

```
## what SSmicmen does, step by step
body(attr(SSmicmen, "initial"))                  # the initial-value routine
## step 1: sort by x and average the replicates
xy <- sortedXyData(pur$conc, pur$rate)
xy
## step 2: Lineweaver-Burk regression of 1/y on 1/x, giving a rough K
lb <- coef(lm(1 / y ~ I(1 / x), data = xy))
K0 <- abs(lb[[2]] / lb[[1]])
K0
## step 3: a partially linear fit of y = Vm * x / (K + x), started at K0
coef(nls(y ~ x / (K + x), data = xy, start = list(K = K0),
         algorithm = "plinear"))
```

```
{
    xy <- sortedXyData(mCall[["input"]], LHS, data)
    if (nrow(xy) < 3) {
        stop("too few distinct input values to fit a Michaelis-Menten model")
    }
    pars <- as.vector(coef(lm(1/y ~ I(1/x), data = xy)))
    pars <- coef(nls(y ~ x/(K + x), data = xy, start = list(K = abs(pars[2L]/pars[1
    L])),
        algorithm = "plinear", ...))
    setNames(pars[c(".lin", "K")], mCall[c("Vm", "K")])
}
     x     y
1 0.02  61.5
2 0.06 102.0
3 0.11 131.0
4 0.22 155.5
5 0.56 196.0
6 1.10 203.5
[1] 0.04362664
           K         .lin
  0.06412123 212.68370727
```

1. `sortedXyData` sorts the data by x and *averages the replicates*: the 12 runs become 6 points $(x, \bar{y})$.

2. A Lineweaver–Burk regression of $1/\bar{y}$ on $1/x$ gives a rough value of K (slope divided by intercept): here $K^{(0)} = 0.044$. This is strategy 2 above.

3. A partially linear fit (Section 6.4) of $\bar{y} = V_m\, x/(K + x)$, started at $K^{(0)}$, needs a starting value for K only, because V_m enters linearly. It returns $(\widehat{V_m}, \widehat{K}) = (212.68,\ 0.0641)$.

Because each concentration has the same number of replicates, least squares on the six averages gives exactly the same estimates as least squares on all twelve runs. So the starting values are already the answer, and `nls` stops immediately. The other built-in self-starters follow the same pattern: a crude linearization or a few summary values of the data, then a partially linear fit. For example `SSlogis` rescales y to the interval $(0, 1)$, takes the logit, regresses x on it to get rough values of the midpoint and the scale, and then fits the logistic curve with the asymptote as the linear parameter.

Writing your own self-starter. Suppose we fit the BOD model $\theta_1(1 - e^{-\theta_2 t})$ often. For a fixed θ_2 the model is linear in θ_1, with least-squares solution

$$\widehat{\theta}_1(\theta_2) = \frac{\sum_i y_i g_i}{\sum_i g_i^2}, \qquad g_i = 1 - e^{-\theta_2 t_i}.$$

So an initial-value routine can search a grid of values of θ_2 only, computing θ_1 in closed form at each, and return the best pair (strategies 4 and 3 combined).

```
## a self-starter for the BOD model th1 * (1 - exp(-th2 * t))
bodModel <- function(t, th1, th2) th1 * (1 - exp(-th2 * t))
bodInit  <- function(mCall, data, LHS, ...) {
  xy  <- sortedXyData(mCall[["t"]], LHS, data)
  th2 <- exp(seq(log(0.01), log(10), length.out = 200))  # grid for th2
  ## for fixed th2, th1 is a least-squares slope: profile it out
  prof <- sapply(th2, function(b) {
    g <- 1 - exp(-b * xy$x)
    sum((xy$y - sum(xy$y * g) / sum(g^2) * g)^2)
  })
  b  <- th2[which.min(prof)]
  g  <- 1 - exp(-b * xy$x)
  setNames(c(sum(xy$y * g) / sum(g^2), b), mCall[c("th1", "th2")])
}
SSbod <- selfStart(bodModel, bodInit, parameters = c("th1", "th2"))

getInitial(demand ~ SSbod(Time, th1, th2), data = BOD)
coef(nls(demand ~ SSbod(Time, th1, th2), data = BOD))  # no start needed
```

```
       th1        th2
19.2276102  0.5231099
       th1        th2
19.1426081  0.5310882
```

Line by line: the mean function `bodModel` has the explanatory variable first and the parameters after it. The initial-value routine `bodInit` receives the matched call `mCall`, the data and the response `LHS` from `nls`; `sortedXyData(mCall[["t"]], LHS, data)` extracts the (x, y) pairs, sorted and averaged; a grid of 200 values of θ_2 between 0.01 and 10 (equally spaced on the log scale) is searched with θ_1 profiled out; and `setNames(..., mCall[c("th1", "th2")])` labels the result with whatever names the user wrote in the formula. Finally `selfStart()` ties the pieces together. The starting values (19.23, 0.523) are close to the estimate (19.14, 0.531), and the new model can be fitted to any data set of this kind with `nls(y ~ SSbod(t, th1, th2), data)`.

Remark 2. A self-starter encodes assumptions about the shape of the data. `SSmicmen`, for instance, needs at least three distinct concentrations, and its Lineweaver–Burk step needs positive responses. A self-starter can fail, or return poor values, on data that do not look like the model; it is a convenience, not a guarantee, and the fitted curve should still be plotted over the data.

6.4 Conditionally linear parameters

Many models are linear in some of their parameters: $\mathbf{f}(\boldsymbol{\theta}) = \mathbf{A}(\boldsymbol{\phi})\,\boldsymbol{\beta}$, where $\boldsymbol{\phi}$ holds the non-linear parameters. For fixed $\boldsymbol{\phi}$ the best $\boldsymbol{\beta}$ is the least-squares solution $\widehat{\boldsymbol{\beta}}(\boldsymbol{\phi}) =$

$\{\mathbf{A}(\boldsymbol{\phi})^\mathsf{T}\mathbf{A}(\boldsymbol{\phi})\}^{-1}\mathbf{A}(\boldsymbol{\phi})^\mathsf{T}\mathbf{y}$, and it remains to minimize the profile sum of squares $\|\mathbf{y} - \mathbf{A}(\boldsymbol{\phi})\widehat{\boldsymbol{\beta}}(\boldsymbol{\phi})\|^2$ over $\boldsymbol{\phi}$ alone. This is the *variable projection* method of Golub and Pereyra (1973): the same minimizer, fewer non-linear parameters, typically fewer iterations and fewer failed starts. In the Michaelis–Menten model V_m is linear given K; in the BOD model θ_1 is linear given θ_2. In R, `nls(..., algorithm = "plinear")` implements the method; the right-hand side of the formula is the model with the linear parameter removed, and only the non-linear parameters need starting values.

```
## Vm enters linearly: only K needs a starting value
fp <- nls(rate ~ conc / (K + conc), data = pur, start = list(K = 0.03),
          algorithm = "plinear")
coef(fp)                                          # .lin is the linear parameter Vm
```

```
          K        .lin
 0.06412126 212.68372966
```

7 When to stop

Common stopping rules are a small relative change in S, a small relative change in $\boldsymbol{\theta}$, or a small gradient $\|\mathbf{J}^\mathsf{T}\mathbf{r}\|$. All of them depend on the units of the problem, and a slow algorithm can satisfy the first two far from the solution: look again at unscaled gradient descent, or at gradient descent stuck on the plateau of the one-parameter example.

Bates and Watts (1981) proposed a criterion with a statistical meaning. Let $\mathbf{J} = \mathbf{Q}_1\mathbf{R}$ be the QR decomposition of the Jacobian at the current iterate, and complete $\mathbf{Q}_1$ to an orthogonal matrix $[\mathbf{Q}_1\ \mathbf{Q}_2]$. The *relative offset* is

$$\text{offset} = \frac{\|\mathbf{Q}_1^\mathsf{T}\mathbf{r}\|/\sqrt{p}}{\|\mathbf{Q}_2^\mathsf{T}\mathbf{r}\|/\sqrt{n-p}}.$$

The numerator measures the part of the residual vector that lies in the tangent plane of the expectation surface, which could still be removed by moving $\boldsymbol{\theta}$; the denominator measures the part orthogonal to it, which is noise. At the solution $\mathbf{Q}_1^\mathsf{T}\mathbf{r} = \mathbf{0}$ by the normal equations. The offset is free of units, and we stop when the improvement still available is tiny compared with the noise.

R's `nls` uses the same idea but omits the constant factor $\sqrt{(n-p)/p}$, so its values are smaller by that factor, here $\sqrt{10/2} = 2.24$. With `trace = TRUE`, `nls` prints S, its convergence measure in parentheses, and the parameters at each iteration.

```
fm <- nls(rate ~ Vm * conc / (K + conc), data = pur,
          start = list(Vm = 250, K = 0.03), trace = TRUE)
unlist(nls.control()[c("maxiter", "tol", "minFactor")])
```

```
35495.36    (4.61e+00): par = (250 0.03)
2013.275    (7.71e-01): par = (205.6133 0.0464387)
1226.002    (1.52e-01): par = (210.524 0.05996853)
1195.876    (1.80e-02): par = (212.3758 0.0636047)
1195.453    (1.78e-03): par = (212.651 0.06406983)
1195.449    (1.72e-04): par = (212.6806 0.06411631)
1195.449    (1.65e-05): par = (212.6834 0.0641208)
```

```
1195.449    (1.59e-06): par = (212.6837 0.06412124)
     maxiter          tol    minFactor
5.000000e+01 1.000000e-05 9.765625e-04
```

The iterates agree line by line with the Gauss–Newton table of Section 5. The default tolerance is 10^{-5}, at most 50 iterations are allowed, and the step factor may be halved down to 1/1024.

8 Summary

Method	Needs	Local rate	Main weakness
Gradient descent	∇S	linear, governed by the condition number	slow; sensitive to units
Newton	∇S, $\nabla^2 S$	quadratic	second derivatives; fails where $\nabla^2 S$ is not positive definite
Gauss–Newton	$\mathbf{J}$	linear with factor ρ; quadratic if $\mathbf{r} = \mathbf{0}$	full steps can diverge; needs $\mathbf{J}$ of full rank
Gauss–Newton with step halving	$\mathbf{J}$	as Gauss–Newton	stalls if $\mathbf{J}$ is (nearly) rank-deficient

- $\nabla S = -2\mathbf{J}^\mathsf{T}\mathbf{r}$ and $\nabla^2 S = 2(\mathbf{J}^\mathsf{T}\mathbf{J} - \sum_i r_i \nabla^2 f_i)$.
- Steepest descent follows $-\nabla S$ (Cauchy–Schwarz), but its speed is governed by the condition number, and it depends on the units of the parameters.
- Newton's method uses curvature and converges quadratically near the solution, but it can head for a maximum or a saddle far from it.
- Gauss–Newton drops the second-derivative term: each step is a linear regression of the residuals on the Jacobian, it always points downhill, and with step halving (the default in `nls`) it is robust.
- The columns of the Jacobian are sensitivities; looking at them explains most failures.
- Good starting values matter as much as the algorithm. Self-starting models compute them from the data, usually by a linearization followed by a partially linear fit.

Exercises

1. Derive (3), and show that $\nabla^2 S = 2\mathbf{X}^\mathsf{T}\mathbf{X}$ when the model is linear. Verify the function `hessS` by deriving the three second derivatives of $V_m x/(K + x)$.
2. In the proof of the Cauchy–Schwarz inequality, why is $\lambda = \mathbf{a}^\mathsf{T}\mathbf{b}/\|\mathbf{b}\|^2$ the minimizer of $\|\mathbf{a} - \lambda\mathbf{b}\|^2$? Show that the contour of S through $\boldsymbol{\theta}$ is perpendicular to $\nabla S(\boldsymbol{\theta})$.

3. For the one-parameter example, compute by hand one gradient-descent step with $t = 0.5$, one Newton step and one Gauss–Newton step from $\theta^{(0)} = 1$, using the table printed in Section 2.1. Which is closest to $\widehat{\theta}$?

4. Using the functions of this handout, count how many steps gradient descent from A needs to reach $S < 1196$ with $\mathbf{D} = \mathrm{diag}(100, 0.03)$. Try a few other choices of $\mathbf{D}$ and compute the condition number of $\mathbf{D}\nabla^2 S(\widehat{\boldsymbol{\theta}})\mathbf{D}$ for each. Which $\mathbf{D}$ makes the condition number smallest?

5. Show that for the BOD model the Gauss–Newton step from $(20, 5)$ is (nearly) the least-squares fit of a regression whose second regressor is almost zero, and explain why its coefficient is unstable.

6. Write a self-starter for the exponential decay model $y_0 e^{-kt}$ whose initial-value routine regresses $\log y$ on t. (Exclude non-positive responses.) Try it on the terminal phase ($t \geq 6$ hours) of subject 1 of the built-in `Theoph` data.

7. Explain why the relative offset is unchanged if every y_i is multiplied by 10 (with the model rescaled accordingly). Is the same true of the rule $|S^{(k)} - S^{(k+1)}| < 10^{-6}$?

References

Armijo, L. (1966). Minimization of functions having Lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1), 1–3.

Bates, D. M. and Watts, D. G. (1981). A relative offset orthogonality convergence criterion for nonlinear least squares. *Technometrics*, 23(2), 179–183.

Bates, D. M. and Watts, D. G. (1988). *Nonlinear Regression Analysis and Its Applications*. Wiley, New York.

Dennis, J. E. and Schnabel, R. B. (1996). *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. SIAM, Philadelphia. (Reprint of the 1983 Prentice-Hall edition.)

Golub, G. H. and Pereyra, V. (1973). The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10(2), 413–432.

Hartley, H. O. (1961). The modified Gauss–Newton method for the fitting of non-linear regression functions by least squares. *Technometrics*, 3(2), 269–280.

Lineweaver, H. and Burk, D. (1934). The determination of enzyme dissociation constants. *Journal of the American Chemical Society*, 56(3), 658–666.

Nocedal, J. and Wright, S. J. (2006). *Numerical Optimization*, 2nd ed. Springer, New York.

Pinheiro, J. C. and Bates, D. M. (2000). *Mixed-Effects Models in S and S-PLUS*. Springer, New York.

Ritz, C. and Streibig, J. C. (2008). *Nonlinear Regression with R*. Springer, New York.

Treloar, M. A. (1974). *Effects of Puromycin on Galactosyltransferase in Golgi Membranes*. M.Sc. thesis, University of Toronto.